

TUGAS PRAKTIKUM 4

SISTEM OPERASI



MUHAMMAD MAULANA NUR SULAIMAN

2410651033

UNIVERITAS MUHAMMDIYAH JEMBER

FAKULTAS TEKNIK

PROGRAM STUDI TEKNIK INFORMATIKA

2025

Tugas 1: Eksperimen dengan Data Berbeda

Jalankan setiap algoritma dengan data berikut dan catat hasilnya:

Data Set 1:

P1: AT=0, BT=10

P2: AT=1, BT=5

P3: AT=2, BT=8

Data Set 2:

P1: AT=0, BT=5

P2: AT=1, BT=3

P3: AT=2, BT=8

P4: AT=3, BT=6

a. Buat file program baru:

nano tugas1.c

```
#include <stdio.h>
```

```
struct Process {
```

```
    int pid;
```

```
    int at; // Arrival Time
```

```
    int bt; // Burst Time
```

```
    int ct; // Completion Time
```

```
    int tat; // Turnaround Time
```

```
    int wt; // Waiting Time
```

```
};
```

// Fungsi untuk menampilkan hasil

```
void printResult(struct Process p[], int n) {  
  
    float avg_tat = 0, avg_wt = 0;  
  
    printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");  
  
    for (int i = 0; i < n; i++) {  
  
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n", p[i].pid, p[i].at, p[i].bt, p[i].ct, p[i].tat,  
p[i].wt);  
  
        avg_tat += p[i].tat;  
  
        avg_wt += p[i].wt;  
  
    }  
  
    printf("\nRata-rata Turnaround Time = %.2f", avg_tat / n);  
  
    printf("\nRata-rata Waiting Time    = %.2f\n", avg_wt / n);  
  
}
```

// Algoritma FCFS (First Come First Serve)

```
void FCFS(struct Process p[], int n) {  
  
    printf("\n=== Algoritma FCFS ===\n");  
  
    int time = 0;  
  
    for (int i = 0; i < n; i++) {  
  
        if (time < p[i].at) time = p[i].at; // tunggu jika proses belum datang  
  
        time += p[i].bt;  
  
        p[i].ct = time;  
  
        p[i].tat = p[i].ct - p[i].at;  
  
        p[i].wt = p[i].tat - p[i].bt;  
  
    }  
  
}
```

```

    }

    printResult(p, n);
}

int main() {

    // ===== Dataset 1 =====

    struct Process data1[] = {

        {1, 0, 10},

        {2, 1, 5},

        {3, 2, 8}

    };

    int n1 = 3;

    // ===== Dataset 2 =====

    struct Process data2[] = {

        {1, 0, 5},

        {2, 1, 3},

        {3, 2, 8},

        {4, 3, 6}

    };

    int n2 = 4;


    printf("DATA SET 1:");

    FCFS(data1, n1);

```

```

printf("\n\nDATA SET 2:");

FCFS(data2, n2);

return 0;

}

```

Penjelasan:

Fungsi Utama: FCFS()

- Inisialisasi time = 0
- Untuk setiap proses:
 1. Jika proses belum datang ($\text{time} < \text{at}$), maka CPU menunggu.
 2. Jalankan proses sesuai burst time.
 3. Hitung nilai:
 - $\text{CT} = \text{time selesai}$
 - $\text{TAT} = \text{CT} - \text{AT}$
 - $\text{WT} = \text{TAT} - \text{BT}$

Fungsi printResult()

Menampilkan hasil setiap proses dan menghitung rata-rata:

- Rata-rata Turnaround Time (TAT)
- Rata-rata Waiting Time (WT)

- b. Compile dan jalankan


```
gcc tugas1.c -o tugas1 -pthread
./tugas1
```

```

alangenteng@LAPTOP-CV1QG677:~$ nano tugas1.c
alangenteng@LAPTOP-CV1QG677:~$ gcc tugas1.c -o tugas1 -pthread
alangenteng@LAPTOP-CV1QG677:~$ ./tugas1
DATA SET 1:
=== Algoritma FCFS ===

PID    AT    BT    CT    TAT    WT
P1      0     10    10     10     0
P2      1      5    15     14     9
P3      2      8    23     21    13

Rata-rata Turnaround Time = 15.00
Rata-rata Waiting Time    = 7.33

DATA SET 2:
=== Algoritma FCFS ===

PID    AT    BT    CT    TAT    WT
P1      0      5      5      5      0
P2      1      3      8      7      4
P3      2      8     16     14      6
P4      3      6     22     19     13

Rata-rata Turnaround Time = 11.25
Rata-rata Waiting Time    = 5.75
alangenteng@LAPTOP-CV1QG677:~$ █

```

Tugas 2: Analisis Eksperimen ketiga algoritma

Jawab pertanyaan berikut:

1. Algoritma mana yang memberikan Average Waiting Time terkecil?
2. Mengapa hasil algoritma berbeda-beda?
3. Kapan sebaiknya menggunakan algoritma FCFS?
4. Kapan sebaiknya menggunakan algoritma SJF?
5. Kapan sebaiknya menggunakan algoritma Round Robin?

Jawab:

1. Algoritma mana yang memberikan Average Waiting Time terkecil?
 - Biasanya SJF memberikan Average Waiting Time (AWT) terkecil.
2. Mengapa hasil algoritma berbeda-beda?
 - Karena strategi penjadwalan berbeda (urutan kedatangan, durasi terpendek, atau pembagian quantum).

3. Kapan sebaiknya menggunakan FCFS?
 - Sistem batch dengan durasi proses mirip, ketika kesederhanaan diutamakan.
4. Kapan sebaiknya menggunakan SJF?
 - Ketika burst time dapat diprediksi, dan tujuan meminimalkan AWT.
5. Kapan sebaiknya menggunakan Round Robin?
 - Sistem interaktif / multitasking, butuh respon cepat dan fairness.

Tugas 3: Modifikasi Program

Tambahkan fitur berikut pada salah satu program:

1. Tampilkan Gantt Chart yang lebih detail
2. Simpan hasil ke file text
3. Baca input dari file
 - a. Buat file program:
Nano tugas1_modif.c

```
#include <stdio.h>
```

```
struct Process {  
  
    int pid; // Process ID  
  
    int at; // Arrival Time  
  
    int bt; // Burst Time  
  
    int ct; // Completion Time  
  
    int tat; // Turnaround Time  
  
    int wt; // Waiting Time
```

```

};

// Fungsi untuk menampilkan hasil dan juga menyimpan ke file

void printAndSaveResult(struct Process p[], int n) {

    FILE *f = fopen("hasil_fcfs.txt", "w");

    if (f == NULL) {

        printf("Gagal membuka file!\n");

        return;

    }

    float avg_tat = 0, avg_wt = 0;

    printf("\n\n=== HASIL PENJADWALAN FCFS ===\n");

    printf("PID\tAT\tBT\tCT\tTAT\tWT\n");

    fprintf(f, "PID\tAT\tBT\tCT\tTAT\tWT\n");

    for (int i = 0; i < n; i++) {

        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",

            p[i].pid, p[i].at, p[i].bt, p[i].ct, p[i].tat, p[i].wt);

        fprintf(f, "P%d\t%d\t%d\t%d\t%d\t%d\n",

            p[i].pid, p[i].at, p[i].bt, p[i].ct, p[i].tat, p[i].wt);

        avg_tat += p[i].tat;

        avg_wt += p[i].wt;
    }

```



```

    }

    avg_tat /= n;

    avg_wt /= n;


    printf("\nRata-rata Turnaround Time = %.2f", avg_tat);

    printf("\nRata-rata Waiting Time    = %.2f\n", avg_wt);


    fprintf(f, "\nRata-rata Turnaround Time = %.2f", avg_tat);

    fprintf(f, "\nRata-rata Waiting Time    = %.2f\n", avg_wt);


    fclose(f);

    printf("\n>> Hasil juga disimpan di file: hasil_fcfs.txt\n");

}

// Fungsi untuk menampilkan Gantt Chart

void printGanttChart(struct Process p[], int n) {

    printf("\n\n=== GANTT CHART ===\n");

    printf(" ");

    for (int i = 0; i < n; i++) {

        for (int j = 0; j < p[i].bt; j++)

            printf("--");

        printf(" ");
    }

```

```

}

printf("\n|");

for (int i = 0; i < n; i++) {

    int mid = p[i].bt;

    for (int j = 0; j < mid - 1; j++)

        printf(" ");

    printf("P%d", p[i].pid);

    for (int j = 0; j < mid - 1; j++)

        printf(" ");

    printf("|");

}

```

```

printf("\n ");

for (int i = 0; i < n; i++) {

    for (int j = 0; j < p[i].bt; j++)

        printf("--");

    printf(" ");

}

```

```

printf("\n0");

for (int i = 0; i < n; i++) {

    printf("%*d", p[i].bt * 2, p[i].ct);

```

```

    }

    printf("\n");
}

// Algoritma FCFS

void FCFS(struct Process p[], int n) {

    int time = 0;

    for (int i = 0; i < n; i++) {

        if (time < p[i].at) time = p[i].at;

        time += p[i].bt;

        p[i].ct = time;

        p[i].tat = p[i].ct - p[i].at;

        p[i].wt = p[i].tat - p[i].bt;

    }

    printAndSaveResult(p, n);

    printGanttChart(p, n);

}

int main() {

    // Baca dari input.txt (format: pid at bt per baris)

    struct Process p[20];

    int n = 0;

```

```

FILE *f = fopen("input.txt", "r");

if (f == NULL) {

    printf("File input.txt tidak ditemukan! Menggunakan data default.\n");

    struct Process p_default[] = {

        {1,0,10}, {2,1,5}, {3,2,8}

    };

    n = 3;

    for (int i=0;i<n;i++) p[i] = p_default[i];

} else {

    while (fscanf(f, "%d %d %d", &p[n].pid, &p[n].at, &p[n].bt) == 3) n++;

    fclose(f);

}

FCFS(p, n);

return 0;

}

```

- Membaca data dari file (input.txt)
- Menyimpan hasil ke file (hasil_fcfs.txt)
- Menampilkan Gantt Chart

```

alangenteng@LAPTOP-CV1QG677:~$ nano tugas1_modif.c
alangenteng@LAPTOP-CV1QG677:~$ gcc tugas1_modif.c -o tugas1_modif pthread
/usr/bin/ld: cannot find pthread: No such file or directory
collect2: error: ld returned 1 exit status
alangenteng@LAPTOP-CV1QG677:~$ gcc tugas1_modif.c -o tugas1_modif
alangenteng@LAPTOP-CV1QG677:~$ ./tugas1_modif
File input.txt tidak ditemukan! Menggunakan data default.

=== HASIL PENJADWALAN FCFS ===
PID    AT    BT    CT    TAT    WT
P1      0    10    10    10     0
P2      1     5    15    14     9
P3      2     8    23    21    13

Rata-rata Turnaround Time = 15.00
Rata-rata Waiting Time    = 7.33

>> Hasil juga disimpan di file: hasil_fcfs.txt

=== GANTT CHART ===
-----
|          P1          |    P2    |    P3    |
-----
0              10        15        23

```

Tugas 4: Eksperimen dengan data berbeda untuk perbandingan algoritma

Test Case 1: Proses dengan burst time bervariasi

Test Case yang diberikan:

Test Case 1 (time quantum=3, 4 proses)

- P1: AT=0, BT=24
- P2: AT=1, BT=3
- P3: AT=2, BT=3
- P4: AT=3, BT=6

Test Case 2: Proses datang bersamaan

Test Case 2 (time quantum=4): semua datang bersamaan (5 proses)

Test Case 3: Proses datang dengan interval

Test Case 3 (time quantum=2): proses datang dengan interval (5 proses)

a. Buat file program:

```
#include <stdio.h>
```

```
struct Process {
```

```
    int pid;
```

```
    int at;
```

```
    int bt;
```

```
    int ct;
```

```
    int tat;
```

```
    int wt;
```

```
    int rt; // remaining time (untuk RR)
```

```
};
```

```
// Cetak hasil
```

```
void printResult(struct Process p[], int n, char *algoname) {
```

```
    float avg_tat = 0, avg_wt = 0;
```

```
    printf("\n=== %s ===\n", algoname);
```

```
    printf("PID\tAT\tBT\tCT\tTAT\tWT\n");
```

```
    for (int i = 0; i < n; i++) {
```

```
        p[i].tat = p[i].ct - p[i].at;
```

```
        p[i].wt = p[i].tat - p[i].bt;
```

```

        avg_tat += p[i].tat;

        avg_wt += p[i].wt;

        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",

                p[i].pid, p[i].at, p[i].bt, p[i].ct, p[i].tat, p[i].wt);

    }

    printf("Rata-rata TAT = %.2f\n", avg_tat / n);

    printf("Rata-rata WT = %.2f\n", avg_wt / n);

}

// FCFS

void FCFS(struct Process p[], int n) {

    int time = 0;

    for (int i = 0; i < n; i++) {

        if (time < p[i].at) time = p[i].at;

        time += p[i].bt;

        p[i].ct = time;

    }

    printResult(p, n, "FCFS");

}

// SJF Non-preemptive

void SJF(struct Process p[], int n) {

    int completed = 0, time = 0;

    int done[10] = {0};

```

```

while (completed < n) {

    int idx = -1, min_bt = 9999;

    for (int i = 0; i < n; i++) {

        if (!done[i] && p[i].at <= time && p[i].bt < min_bt) {

            min_bt = p[i].bt;

            idx = i;

        }

    }

    if (idx == -1) {

        time++;

        continue;

    }

    time += p[idx].bt;

    p[idx].ct = time;

    done[idx] = 1;

    completed++;

}

printResult(p, n, "SJF Non-preemptive");

}

// Round Robin

void RR(struct Process p[], int n, int tq) {

```



```
int completed = 0;
```

```
int time = 0;
```

```
for (int i = 0; i < n; i++)
```

```
    p[i].rt = p[i].bt; // remaining time
```

```
while (completed < n) {
```

```
    int all_done = 1;
```

```
    for (int i = 0; i < n; i++) {
```

```
        if (p[i].rt > 0 && p[i].at <= time) {
```

```
            all_done = 0;
```

```
            if (p[i].rt > tq) {
```

```
                time += tq;
```

```
                p[i].rt -= tq;
```

```
            } else {
```

```
                time += p[i].rt;
```

```
                p[i].rt = 0;
```

```
                p[i].ct = time;
```

```
                completed++;
```

```
            }
```

```
        } else if (p[i].rt > 0 && p[i].at > time) {
```

```
            time++; // tunggu proses berikutnya datang
```

```

        }

    }

    if (all_done) break;

}

printResult(p, n, "Round Robin");

}

// Salin array agar tidak tercampur antar algoritma

void copyData(struct Process src[], struct Process dest[], int n) {

    for (int i = 0; i < n; i++)

        dest[i] = src[i];

}

// Jalankan satu test case

void runTestCase(struct Process data[], int n, int tq, char *judul) {

    printf("\n\n===== \n%s\n =====\n", judul);

    struct Process fcfs[10], sjf[10], rr[10];

    copyData(data, fcfs, n);

    copyData(data, sjf, n);

    copyData(data, rr, n);

    FCFS(fcfs, n);

```

```

    SJF(sjf, n);

    RR(rr, n, tq);

}

int main() {

    // TEST CASE 1

    struct Process tc1[] = {

        {1, 0, 24}, {2, 1, 3}, {3, 2, 3}, {4, 3, 6}

    };

    runTestCase(tc1, 4, 3, "TEST CASE 1: Burst Time Bervariasi");

    // TEST CASE 2

    struct Process tc2[] = {

        {1, 0, 10}, {2, 0, 5}, {3, 0, 8}, {4, 0, 12}, {5, 0, 6}

    };

    runTestCase(tc2, 5, 4, "TEST CASE 2: Semua Proses Datang Bersamaan");

    // TEST CASE 3

    struct Process tc3[] = {

        {1, 0, 8}, {2, 2, 4}, {3, 4, 9}, {4, 6, 5}, {5, 8, 3}

    };

    runTestCase(tc3, 5, 2, "TEST CASE 3: Proses Datang Secara Interval");

    return 0;

}

```

Hasil:

```
alangenteng@LAPTOP-CV1QG677:~$ nano tugas4.c
alangenteng@LAPTOP-CV1QG677:~$ gcc tugas4.c -o tugas4
alangenteng@LAPTOP-CV1QG677:~$ ./tugas4
```

```
=====
TEST CASE 1: Burst Time Bervariasi
=====
```

```
=== FCFS ===
```

PID	AT	BT	CT	TAT	WT
P1	0	24	24	24	0
P2	1	3	27	26	23
P3	2	3	30	28	25
P4	3	6	36	33	27

Rata-rata TAT = 27.75
Rata-rata WT = 18.75

```
=== SJF Non-preemptive ===
```

PID	AT	BT	CT	TAT	WT
P1	0	24	24	24	0
P2	1	3	27	26	23
P3	2	3	30	28	25
P4	3	6	36	33	27

Rata-rata TAT = 27.75
Rata-rata WT = 18.75

```
=== Round Robin ===
```

PID	AT	BT	CT	TAT	WT
P1	0	24	36	36	12
P2	1	3	6	5	2
P3	2	3	9	7	4
P4	3	6	18	15	9

Rata-rata TAT = 15.75
Rata-rata WT = 6.75

```
=====
TEST CASE 2: Semua Proses Datang Bersamaan
=====
```

```
=== FCFS ===
```

PID	AT	BT	CT	TAT	WT
P1	0	10	10	10	0
P2	0	5	15	15	10
P3	0	8	23	23	15
P4	0	12	35	35	23
P5	0	6	41	41	35

Rata-rata TAT = 24.80
Rata-rata WT = 16.60

```
=== SJF Non-preemptive ===
```

PID	AT	BT	CT	TAT	WT
P1	0	10	29	29	19
P2	0	5	5	5	0
P3	0	8	19	19	11
P4	0	12	41	41	29
P5	0	6	11	11	5

Rata-rata TAT = 21.00
Rata-rata WT = 12.80

```
=== Round Robin ===
```

PID	AT	BT	CT	TAT	WT
P1	0	10	37	37	27
P2	0	5	25	25	20
P3	0	8	29	29	21
P4	0	12	41	41	29
P5	0	6	35	35	29

Rata-rata TAT = 33.40
Rata-rata WT = 25.20

```

=====
TEST CASE 3: Proses Datang Secara Interval
=====

=== FCFS ===
PID    AT    BT    CT    TAT    WT
P1      0     8     8     8     0
P2      2     4    12    10     6
P3      4     9    21    17     8
P4      6     5    26    20    15
P5      8     3    29    21    18
Rata-rata TAT = 15.20
Rata-rata WT  = 9.40

=== SJF Non-preemptive ===
PID    AT    BT    CT    TAT    WT
P1      0     8     8     8     0
P2      2     4    15    13     9
P3      4     9    29    25    16
P4      6     5    20    14     9
P5      8     3    11     3     0
Rata-rata TAT = 12.60
Rata-rata WT  = 6.80

=== Round Robin ===
PID    AT    BT    CT    TAT    WT
P1      0     8    26    26    18
P2      2     4    14    12     8
P3      4     9    29    25    16
P4      6     5    24    18    13
P5      8     3    19    11     8
Rata-rata TAT = 18.40
Rata-rata WT  = 12.60

```

Tugas 5: Analisis Eksperimen perbandingan algoritma

Setelah menjalankan program dengan berbagai test case, jawab pertanyaan berikut:

1. Analisis Performa
 - FCFS terbaik ketika proses memiliki burst time mirip dan sistem batch.
 - SJF hampir selalu memberikan Average WT terkecil karena memilih job terpendek lebih dulu (optimal untuk AWT jika durasi diketahui).
 - Trade-off RR: adil dan responsif tetapi banyak context switch jika quantum kecil.
2. Context Switching
 - RR paling banyak melakukan context switch.
 - Quantum kecil -> banyak context switch; quantum besar -> sedikit context switch.

3. Fairness

- RR paling adil.
- FCFS merugikan proses pendek jika ada proses panjang di depan (convoy effect).
- SJF berisiko starvation untuk proses panjang.

4. Real World Application

- Web server (many small requests): SJF atau RR dengan quantum kecil
- Render video (task besar): FCFS atau SJF.
- OS desktop (interactive): Round Robin.